

Amlogic NPU User Guide

Version: v0.1

Release Date: 2026-05-31

Copyright

© 2026 Amlogic Semiconductor (Shanghai) Co., Ltd. (representing itself and its affiliates, hereinafter "Amlogic") All rights reserved. No part of this document may be reproduced, distributed, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 and other icons of Amlogic are trademarks of Amlogic. All other trademarks and registered trademarks are the property of their respective owners.

Disclaimer

This document may be updated or modified from time to time without notice due to product version upgrades or other reasons.

This product is not suitable for medical, life-saving, or life-sustaining applications, nor for applications where a reasonably foreseeable failure or malfunction of the Amlogic product could lead to personal injury, death, severe property, or environmental damage.

The circuit diagrams and other information related to Amlogic products provided herein are for typical application descriptions only. Complete and sufficient information for production design may not always be provided. Amlogic has endeavored to ensure the accuracy and timeliness of the content in this document but makes no warranties regarding the accuracy or completeness of the information contained herein.

Amlogic products may contain unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic assumes no liability for any vulnerabilities.

The use of related materials may require permission from third-party or Amlogic's intellectual property rights. This document does not grant any express or implied declaration or warranty of intellectual property rights belonging to Amlogic or any other party. Any third-party website links provided in this document are for user convenience only. Amlogic does not endorse such websites or any of their actions (including privacy practices, availability, and content) and assumes no responsibility.

Under no circumstances shall Amlogic be liable for any claims arising from or related to the use or inability to use this document.

Contact Information

- Company Website: www.amlogic.cn
- Pre-sales Inquiry: contact@amlogic.com
- Technical Support: support@amlogic.com

Revision History

Version	Modification Description
v0.1 (2026-05-31)	This is the First Version.

Contents

1 Introduction to Amlogic NPU

- 1.1 Introduction to the Amlogic NPU Software Stack
- 1.2 Amlogic NN ToolChain Introduction
- 1.3 Integration Process Introduction
- 1.4 Integration Steps
- 1.5 Supported Hardware Platforms
- 1.6 Keyword Definitions

2 Development Environment Preparation

- 2.1 Development Environment Verification
 - 2.1.1 DDK Version Verification
 - 2.1.2 System Environment Verification
 - 2.1.3 Platform Type Verification
 - 2.1.4 Integration Method Selection
- 2.2 Toolkit Installation
 - 2.2.1 Installation Introduction
 - 2.2.2 Toolkit Installation
 - 2.2.2.1 Configuring Virtual Environment
 - 2.2.2.2 Installing Toolkit
 - 2.2.3 Native C Compilation Environment Installation

3 Model Conversion

- 3.1 Model Conversion Introduction
- 3.2 Model Conversion Preparation
 - 3.2.1 Model Information Verification
 - 3.2.2 Quantization Dataset Preparation
- 3.3 Model Conversion
 - 3.3.1 AMLNN Initialization and Release
 - 3.3.2 Model Loading
 - 3.3.3 Model Conversion Parameter Configuration
 - 3.3.4 Model Compilation
 - 3.3.5 Exporting ADLA Model
 - 3.3.6 Model Conversion Example
- 3.4 Model Evaluation
 - 3.4.1 Model Inference
 - 3.4.2 Model Performance Evaluation
 - 3.4.3 Model Accuracy Evaluation
 - 3.4.4 NPU Utilization
- 3.5 On-device C API Inference
 - 3.5.1 Process Introduction

4 On-device Python API Inference

- 4.1 Python API Process Introduction
- 4.2 Python API Description
 - 4.2.1 AMLNNLite Initialization
 - 4.2.2 init_runtime
 - 4.2.3 get_sdk_version
 - 4.2.4 load_model
 - 4.2.5 get_tensor_info
 - 4.2.6 inference
 - 4.2.7 uninit
 - 4.2.8 Python API Example Demo

1 Introduction to Amlogic NPU

Amlogic NPU is a software solution for AI model deployment introduced by the Amlogic NN Team, designed to provide developers and customers with a complete edge computing model deployment platform. It covers the entire workflow from model conversion to on-device inference, helping users quickly evaluate the feasibility and performance of models on the Amlogic NN platform, thereby shortening the overall development cycle.

1.1 Introduction to the Amlogic NPU Software Stack

The software stack architecture diagram is as follows:

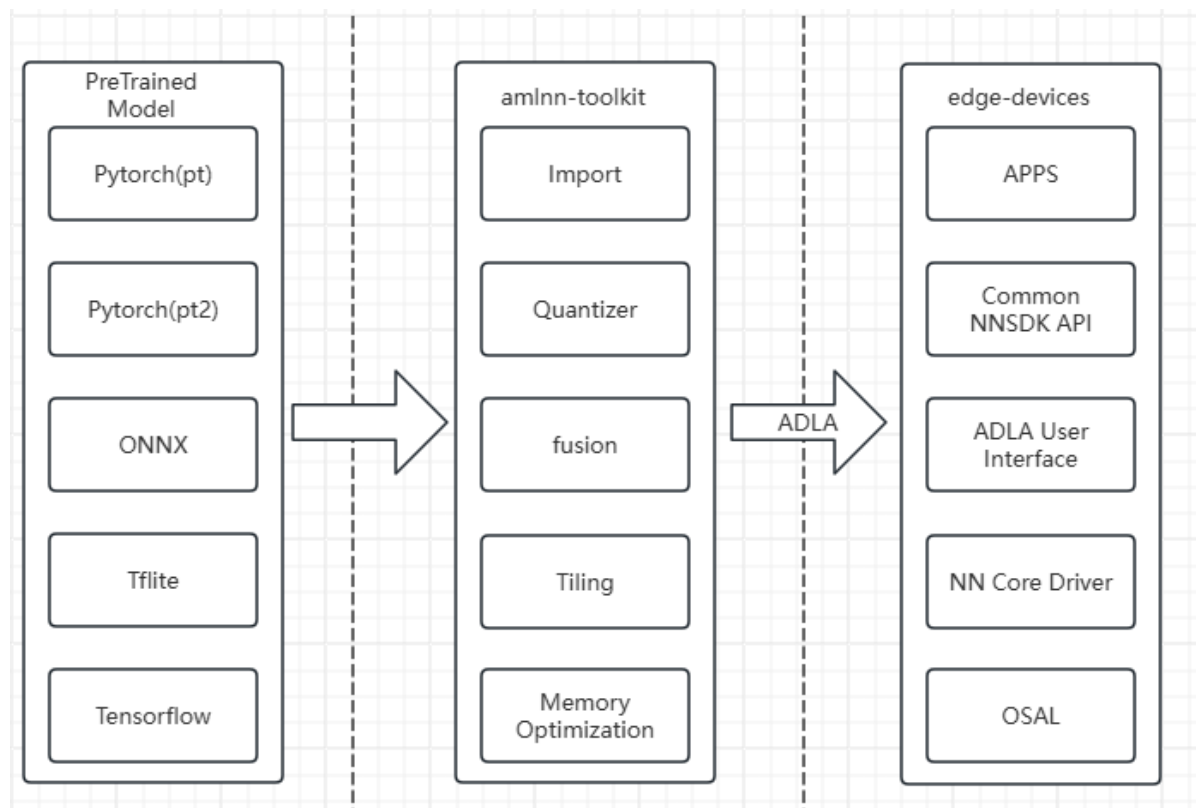


Figure 1.1-NN_WorkFlow

1.2 Amlogic NN ToolChain Introduction

- **ADLA Convert ToolChain:** This component supports importing and compiling models from frameworks such as PyTorch, ONNX, and TFLite into ADLA model files in Ubuntu PC or on-device Debian environments. It supports loading and inference of ADLA models via Python API by connecting to the device via ADB or directly within the Debian on-device environment. The current release includes four types of whl packages, described as follows:
 - **amlnn_toolkit**
 - Support Model quantization conversion, exporting ADLA on PC
 - Support ADLA model inferencing by Connecting the board via ADB
 - Support GGUF(LLM/VLM) quantization conversion (Refer to LLM documentation)
 - **amlnn_edge_toolkit**

- Support Model quantization conversion, exporting ADLA on edge side
- Support ADLA model inferencing by Connecting the board via ADB
- Support ADLA model inferencing on edge side
- **amlInn_edge_toolkit_lite**
 - Support ADLA model inferencing on edge side
- **amlIIm_edge_toolkit_lite**
 - Support ADLA model (LLM/VLM) inferencing on edge side (Refer to LLM documentation)
- **Comparison of features supported by the three whl packages:**

Function List	amlInn_toolkit	amlInn_edge_toolkit	amlInn_edge_toolkit_lite
Environment: System Env	x86_64	aarch64	aarch64
Conversion: Model quantization conversion & exporting ADLA	✓	✓	×
Conversion: Support GGUF(LLM/VLM) conversion & Exporting ADLA	✓	×	×
Inference: Support ADLA model inferencing via ADB	✓	✓	×
Inference: Support ADLA model inferencing on edge side	×	✓	✓

- **ADLA Runtime:** C/C++ inference SDK for Amlogic NPU, providing core capabilities such as model loading, inference execution, and context management. Supports fast integration and efficient on-device deployment. Includes two runtimes: `nn_runtime` (for common models) and `llm_runtime` (for LLM/VLM models).
 - **nn_runtime:** C/C++ inference SDK for the Amlogic NPU platform, used for small model deployment. Integrate into native applications by linking `libnnsdk.so` and including the `nnsdk2.h` header file. Supports Android and Linux (Buildroot / Yocto / Debian) platforms.
 - **llm_runtime:** C/C++ inference SDK for the Amlogic NPU platform, used for large language model deployment. Integrate into native applications by linking `libllmsdk.so` and including the `llmsdk.h` header file. Supports Android and Linux (Yocto / Debian) platforms.

1.3 Integration Process Introduction

Model Integration refers to migrating the model loading and inference pipeline from the Python environment on the PC to run on edge devices and re-implementing operations such as Preprocessing and Postprocessing in C/C++ or Java. The general process is as follows:

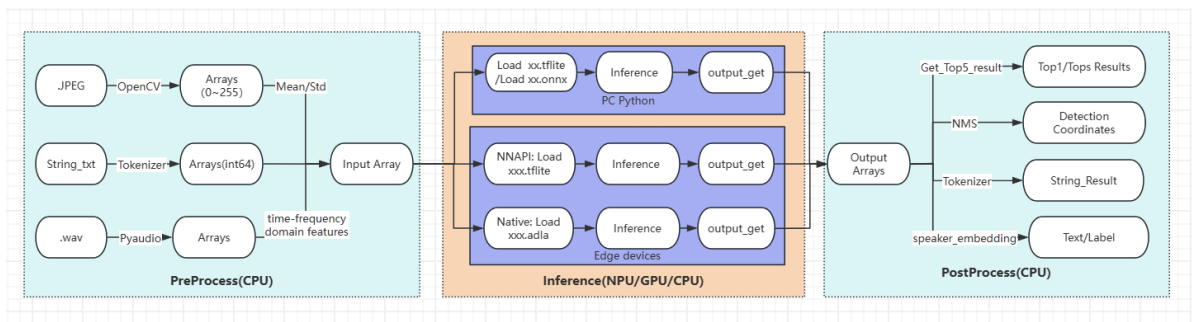


Figure 1.3-Integration Process

- **PreProcess:** When integrating models on edge devices, the preprocessing module needs to be implemented in C/C++ or Java. Typical preprocessing flows are described below:
 - **CV Model Preprocessing:** For image input, the preprocessing module includes:
 - **Acquire Raw Data:** Read an image into a multi-dimensional array in the range [0~255] using OpenCV, or obtain a multi-dimensional array from a camera.
 - **Normalization:** Perform normalization on the raw data based on the mean and standard deviation (std) used during model training. For each channel of input image x : $x(\text{norm}) = (x - \text{mean}) / \text{std}$. This method directly operates on integer pixel values in the range [0, 255], consistent with the processing method when reading images and performing quantization in `amlnn-toolkit`. Note that some frameworks first scale pixels from [0, 255] to [0, 1] (i.e., $x / 255.0$) before performing the normalization operation of subtracting mean and dividing by std. Please convert according to the specific framework requirements.
 - **Input Quantization:** Quantization of normalized data is only required when deploying on edge devices and the generated ADLA model requires fixed-point input. Quantization is performed based on the input quantization parameters (scale, zero_point) of the ADLA model: $x(\text{quant}) = x(\text{norm}) / \text{scale} + \text{zero_point}$
 - **Preprocessing Optimization:** When deploying on edge devices, the formulas for normalization and input quantization can be combined and expanded to reduce redundant computations and effectively lower preprocessing latency.
 - **LLM Chat Model Preprocessing:**
 - Use Tokenizer to convert raw strings into `input_ids` sequences acceptable to the model, while also generating auxiliary inputs like `attention_mask`.
 - **Audio Model Preprocessing:**
 - Read `.wav` files and convert them into numerical arrays to extract time-frequency domain features.
- **Inference:**
 - **Native Method:**
 - **Native(c/c++):** Convert PC-side ONNX, PyTorch, TFLite, or other framework models into ADLA model files, then load them onto the Amlogic NPU for inference using NNSDK interfaces.
 - **Native(Python Api):** Use the Python API of `amlnn_edge_toolkit` to perform model conversion and integration on the device, or use the Python API of `amlnn_edge_toolkit_lite` to integrate ADAL model files converted on the PC.

- **Runtime Method:** In the Android system, load TFLite models through the NNAPI or LiteRT framework, enable Neural Network (NN) acceleration, and finally run the model on the Amlogic NPU.
 - Whether the loaded TFLite model is quantized or floating-point depends on the model formats supported by the Amlogic NPU and the adaptation for inference flows like NNAPI and LiteRT.
- **Third-Party Runtime Method:** Load and run based on third-party Runtime open-source libraries supported and adapted by Amlogic. e.g., ONNXRuntime.
- **PostProcess:** When integrating models on edge devices, the postprocessing module needs to be implemented in C/C++ or Java. Typical postprocessing flows are described below:
 - **Classification Model Postprocessing:** Perform Top-1 or Top-5 processing on the model output results to obtain the corresponding classification categories.
 - **Detection Model Postprocessing:** Decode the model's output bounding boxes and class confidence scores, perform threshold filtering and Non-Maximum Suppression (NMS) to obtain final detection results.
 - **LLM Chat Model Postprocessing:** Decode the token sequences generated by the model into natural language text and remove special tokens such as padding or termination symbols.
 - **Audio Model Postprocessing:** Decode, deduplicate, and reconstruct waveforms from the model's output acoustic features or probability sequences to obtain the final audio or recognized text.

1.4 Integration Steps

The entire integration process consists of the following steps:

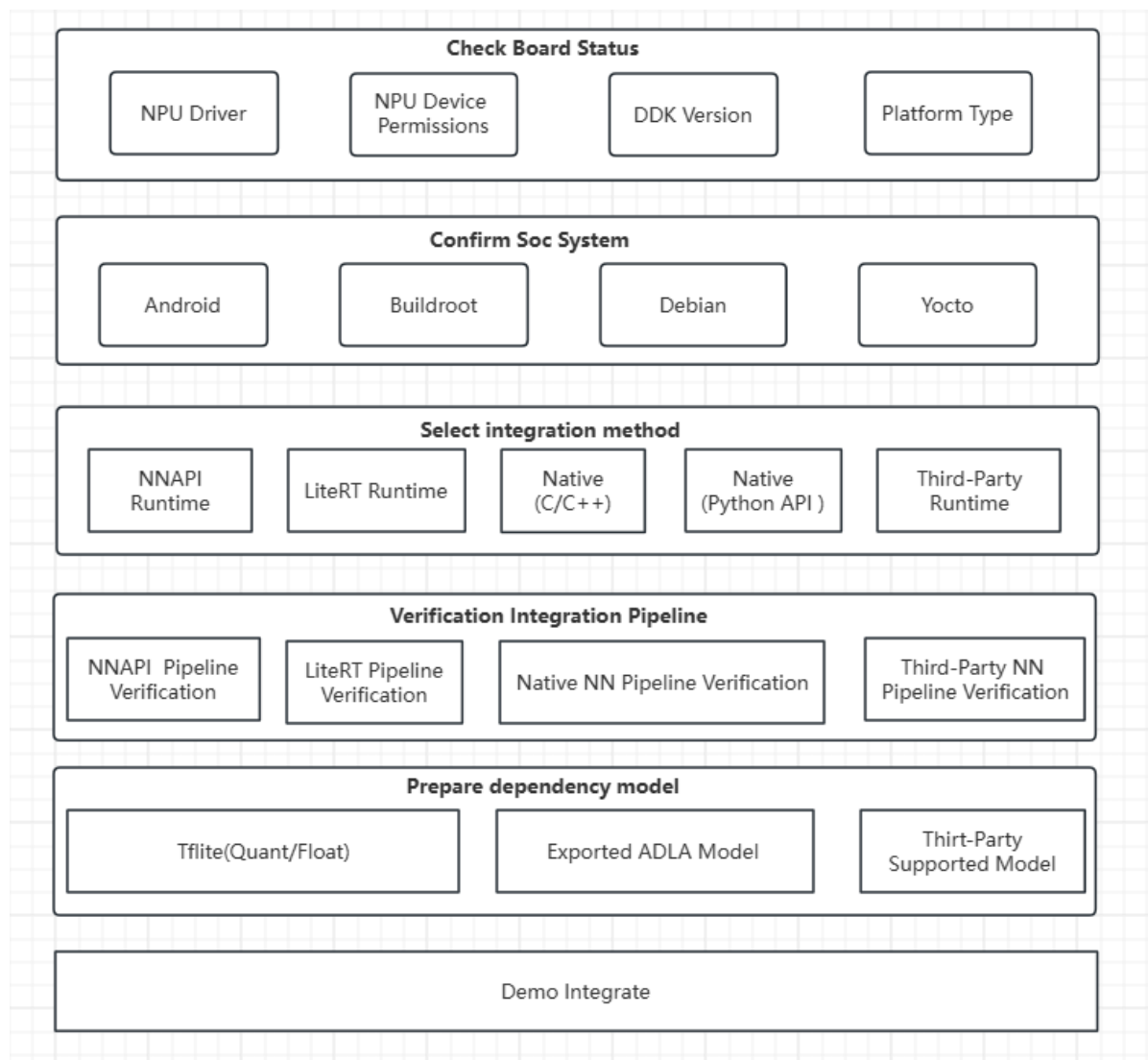


Figure 1.4-Integration Steps

1. Development Environment Verification:

- Verify the NN Driver version on the edge device and confirm the corresponding DDK version based on the NN Driver version.
- Verify the system environment of the edge device [Android / Buildroot / Debian / Yocto].

2. **Integration Method Confirmation:** Choose the integration method based on the edge device's system environment. The integration methods supported by different system environments are as follows:

	Buildroot	Android	Debian	Yocto	Dependency
Native(c/c++ Api)	✓	✓	✓	✓	amlnn_toolkit / ADLA NNSDK
Native(python Api)	×	×	✓	✓	amlnn_edge_toolkit / amlnn_edge_toolkit_lite
NNAPI Runtime	×	✓	×	×	Quant Tflite / Float Tflite
LiteRT Runtime	✓	✓	✓	✓	Quant Tflite / Float Tflite / Litert
Third-Party Runtime	✓	✓	✓	✓	ONNXRuntime

3. **Integration Pipeline Verification:** Quickly verify the feasibility of the selected integration Pipeline.
4. **Model Selection:** Select open-source framework models supported by amlnn-toolkit and export them as standalone deployable model files (containing data flow graph and weights, independent of the original project, and directly loadable by frameworks like ONNX Runtime / TFLite Runtime / PyTorch).
5. **Model Conversion:**
 - Native Integration: Convert open-source framework models into ADLA models corresponding to the target platform.
 - NNAPI / LiteRT Runtime Integration: Convert open-source framework models into Quant TFLite or Float TFLite models.
6. **Model Evaluation:**
 - Native Integration: Evaluate whether the accuracy of the converted ADLA model meets requirements.
 - NNAPI / LiteRT Runtime Integration: Evaluate whether the accuracy of the converted TFLite model meets standards.
7. **Demo Integration:** Complete the development of model preprocessing and postprocessing based on the selected integration method and development environment to establish the entire Demo Pipeline.

1.5 Supported Hardware Platforms

This document is applicable to the following Amlogic hardware platforms:

- C308L / C302X
- S928X
- A311D2
- T968D4
- S905X5 / S905D5
- C302X2
- C305X2 / A123X
- A311Y3

1.6 Keyword Definitions

- **ADLA Model:** ADLA (Amlogic Deep Learning Accelerator), model files running on the Amlogic platform, with the file extension `.adla`.
- **AMLNN:** Amlogic NPU
- **Edge Devices:** Edge devices, specifically referring to Amlogic development boards with NPUs in this document.
- **W4A8 / W8A8 / W4A16 / W8A16 / W16A16:** For example, W4A8 means: weights 4bit, feature 8bit.
- **Native Method:** Does not rely on third-party inference frameworks and requires pre-converting open-source framework models into ADLA model files.
- **Runtime Method:** Relies on third-party inference frameworks; ADLA model files do not need to be pre-converted but are generated during program execution.

2 Development Environment Preparation

2.1 Development Environment Verification

Before starting NN integration, it is necessary to verify the current board environment and then select the integration method. The information to be verified has been introduced in Section 1.4.

2.1.1 DDK Version Verification

First, verify whether the NPU module is loaded correctly and confirm the loaded NPU Driver version.

- **Driver Loading Verification:** Connect to the board via adb shell or directly through the board's command window.
 - **Loading Verification:** Execute: `lsmod | grep adla_core`. The result should contain `adla_core`, as shown below:

```
t7 an400:/ $ lsmod | grep adla_core
adla_core                221184  0
amlogic_cpufreq          16384  18 amwdec_av1_t5d_v41, amwdec_
```

- **Permission Verification:** Execute: `ls -l /dev/adla0`. The permissions should be displayed as 666, as shown below:

```
t7 an400:/ $ ls -l /dev/adla0
crw-rw-rw- 1 system system 463, 0 1970-01-01 00:00 /dev/adla0
```

- **DDK Version Verification:**
 - **Driver Version Verification:** Restart the board and execute the command in the serial port or CMD window after adb shell connection: `dmesg | grep adla`. Obtain logs similar to the following:

```
5.405204] adla kmd version: 0.10.7.1
```

NPU Driver version is: 0.10.7 (obtain the first three digits of the version number).

- **C Runtime Version Verification:** The following commands are supported on the device side:

```
# Android 32-bit execute command
strings /vendor/lib/libnnsdk.so | grep "NNSDK2 Version"

# Android 64-bit execute command
strings /vendor/lib64/libnnsdk.so | grep "NNSDK2 Version"

# Linux (Buildroot 32 / Buildroot 64 / Yocto 32 / Yocto 64 / Debian 64 /
Ubuntu 64) execute command
strings /usr/lib/libnnsdk.so | grep "NNSDK2 Version"

# Prints as follows, version is vx.x.x
NNSDK2 Version: vx.x.x, 20xx.xx
```

○ **DDK Version Verification:**

Confirm the corresponding DDK version based on the NPU Driver version, and ensure the C Runtime and amlnn-toolkit versions match.

DDK Version	NPU Driver Version	C Runtime Version	amlnn-toolkit version
DDK4.7.9.8	1.7.1	2.8.5	adla-toolkit_3.2.9.3
DDK5.10.4.10	2.0.0		amlnn_toolkit

Note: The current document content supports NPU Driver versions greater than 1.7.1.

2.1.2 System Environment Verification

Confirm the system environment and architecture of the board side to provide guidance for selecting the corresponding nserver, and cross-compilation toolchains in subsequent sections.

Step 1: First confirm whether it is an Android platform or a Linux platform

Execute command: adb shell getprop ro.product.cpu.abi

Judgment logic:

- **Output** arm64-v8a: confirmed as 64-bit Android system
- **Output** armeabi-v7a: confirmed as 32-bit Android system
- **Error** command not found: judged as Linux system, please continue to Step 2

Step 2: Further confirmation of Linux system (Buildroot / Yocto / Armbian) environment

Step 2.1: Confirm whether it is an Armbian / Yocto system

Execute command: cat /version.txt

Reference output example (taking Armbian as an example)

```
image: a9-by401-bookworm-20260707030949
BRANCH=
VERSION=Linux/Debian/Armbian-PB#786
SPIN=Linux/Debian/Armbian-PB#786
BUILD_TIME="2026-07-07 03:27:41"
Generated on Tue Jul 7 03:27:41 UTC 2026

Compiler:
aarch64-none-linux-gnu-gcc (Arm GNU Toolchain 11.3.Re11) 11.3.1 20220712
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Judgment logic:

- If this file exists: judged as an Armbian/Yocto system, and then determine whether it is a 32-bit/64-bit system based on the Compiler version information. The reference example corresponds to an Armbian 64-bit (AArch64) system.
- If the file does not exist: command execution fails, judged as a Buildroot system, please continue to Step 2.2

Note: The above confirmation method is only applicable to Amlogic official release boards. Third-party development boards should confirm whether the same version files and judgment logic are supported.

Step 2.2: Confirm Buildroot system architecture

Execute command: adb shell uname -m

Judgment logic:

- `aarch64`: confirmed as 64-bit buildroot system
- `armv7l`: confirmed as 32-bit buildroot system

2.1.3 Platform Type Verification

Before starting the ADLA model conversion process, the board model must first be confirmed to obtain the `target_platform` parameter that fully matches the hardware, avoiding conversion failures or inference abnormalities due to platform incompatibility. Currently, the final `target_platform` value can be queried through the following two independent methods, and all verification results can be checked against the **"Part_number/Serial/target_platform Correspondence Table"** below. The correspondence table is as follows:

Part_number	Serial	target_platform
C308L / C302X	Serial: 3d0a 010100000000c61349	PRODUCT_PID0XA001
S928X	Serial: 3e0a 0202000000007c404b	PRODUCT_PID0XA002
A311D2	Serial: 360c 01030000000025f904	PRODUCT_PID0XA003
T968D4	Serial: 420a 010100000000daff6e	PRODUCT_PID0XA004
S905X5 / S905D5	Serial: 480a 020100000000682763	PRODUCT_PID0XA005
C302X2	Serial: 4c0a 01030000000055665a	PRODUCT_PID0XA006

Part_number	Serial	target_platform
A311Y3	Serial: 4d0a 01020000000070e84a	PRODUCT_PID0XA007
C305X2/A123X	Serial: 4f0a 01010000000054c6d1	PRODUCT_PID0XA008

Method 1: Quick confirmation via Part_number

Directly read the Part_number identifier engraved on the board hardware surface, match the read number with the correspondence table, and obtain the correct target_platform parameter corresponding to the board.

Method 2: Confirmation via Serial number verification

Obtain Serial information: After the board starts normally, connect to the device via ADB and execute the following commands to read the Serial number:

```
adb shell cat /proc/cpuinfo
# or
adb shell cat /proc/cpu_chipid
```

Matching rules:

- **Preferred rule:** Priority is to intercept **the first 4 characters** of the Serial number, compare with the correspondence table to complete matching, and obtain the target `target_platform` value.
- **Alternative rule:** If the first 4 characters have no matching result, you can downgrade to intercepting **the first 3 characters** of the Serial number to complete matching, and finally obtain the correct `target_platform` parameter.

System Environment and NPU Hardware Feature Support for Different Boards:

	Buildroot	Android	Debian	Yocto	INT4	INT8	INT16	F16	BF16
C308L/C302X	✓	×	×	×	×	✓	✓	×	×
S928X	×	✓	×	×	×	✓	✓	×	×
A311D2	×	✓	×	×	×	✓	✓	×	×
T968D4	×	✓	×	×	×	✓	✓	×	×
S905X5/S905D5	×	✓	×	✓	×	✓	✓	×	×
C302X2	✓	×	×	×	×	✓	✓	×	×
A311Y3	×	✓	✓	✓	✓	✓	✓	✓	✓
C305X2/A123X	✓	×	×	×	✓	✓	✓	✓	✓

2.1.4 Integration Method Selection

- Supported Integration Methods:

	Buildroot	Android	Debian	Yocto	Dependency
Native(c/c++ Api)	✓	✓	✓	✓	amlnn_toolkit / ADLA NNSDK
Native(python Api)	×	×	✓	✓	amlnn_edge_toolkit / amlnn_edge_toolkit_lite
NNAPI Runtime	×	✓	×	×	Quant Tflite / Float Tflite
LiteRT Runtime	✓	✓	✓	✓	Quant Tflite / Float Tflite / Litert
Third-Party Runtime	✓	✓	✓	✓	ONNXRuntime

- Integration Method Introduction

- Native(c/c++ Api)

- Use the toolkit to convert and export supported open-source framework models into ADLA models.
- Use Amlogic's unified NNSDK interface to load and infer ADLA models.

- Native (Python Api)

- Method1: Directly use `amlnn_edge_toolkit` to complete ADLA model conversion, loading, and inference on the edge device.
- Method2:
 - First, use the toolkit to convert open-source framework models and export them as ADLA models.
 - Then, use the Python API of `amlnn_edge_toolkit` or `amlnn_edge_toolkit_lite` to load and infer ADLA models.

- NNAPI Runtime

- Documentation Introduction:** [Neural Networks API](#) | [Android NDK](#) | [Android Developers](#)
- Model Preparation:** Prepare **Quant / Float Tflite** models in advance. You Can use `amlnn_toolkit` (including `amlnn_edge_toolkit`) to convert other open-source framework models to Quant Tflite or Float Tflite.
- Platform Support:** Currently, only **A311Y3** supports both **Quant Tflite + Float Tflite**; other platforms only support **Quant Tflite**.
- Support Status:** NNAPI support status across Android system versions is as follows:

Android System	NNAPI
Android R(11) 32bit/64bit	✓
Android T(13) 32bit/64bit	✓
Android U(14) 32bit/64bit	✓
Android B(16) 32bit/64bit	✓

- **LiteRT Runtime**

- All four SOC system environments support LiteRT Runtime. Currently, usage guidance for environments other than Android is pending output.

- **Third-Party Runtime**

- Set up the environment for third-party open-source libraries supported and adapted by Amlogic.
- Complete the loading and inference of supported models (running on Amlogic NPU) based on the third-party environment.

- **Third-Party Runtime Support Status:**

- Currently, only ONNXRuntime is supported. Other third-party runtimes will be supported gradually.

- **Android System: NNAPI Runtime vs Native(c/c++)**

	Advantages	Disadvantages
Native	1. Best Performance: Direct use of NPU interfaces can fully utilize specific hardware capabilities, typically achieving higher performance and lower latency. 2. Flexible Optimization: Developers can perform deep optimization for specific algorithms and models, leveraging NPU characteristics for more efficient implementation. 3. Performance Tuning: Direct access to the NPU allows developers low-level control, facilitating specific performance tuning.	1. Development Complexity: Operating NPU APIs directly increases development and maintenance complexity. 2. Platform Dependency: Different vendors' NPUs have different APIs, potentially preventing code reuse across devices and increasing adaptation costs for multiple platforms.

	Advantages	Disadvantages
NNAPI	1. Cross-Platform Compatibility: NNAPI provides a unified interface, allowing developers to call the same quantized TFLite model across different hardware and devices. 2. Simplified Development Process: Developers do not need to interact with underlying hardware, only need to convert to quantized TFLite, and can focus on model design and application development.	1. Performance Limitations: The NNAPI call flow is relatively long, ultimately converting to NPU-loadable ADLA files at the bottom layer. The first call has online compilation time, which is time-consuming. The call process consumes more memory. 2. Operator Limitations: Not all operations and models are supported by NNAPI, potentially requiring model adjustments to meet NNAPI requirements. 3. Potential Compatibility Issues: Differences in operators supported by NNAPI across Android versions may lead to inconsistent behavior.

2.2 Toolkit Installation

2.2.1 Installation Introduction

Different board system environments support different integration methods, and different integration environments require different toolchains to be installed. First, determine the integration method based on the system environment, then install the corresponding Toolkit.

- **Android System:**

Android	amlnn_toolkit	Native C Compilation Environment
Native(c/c++ Api)	✓	✓
NNAPI Runtime	✓ (Optional)	×
LiteRT Runtime	✓ (Optional)	×
Third-Party Runtime	×	×

- **NNAPI / LiteRT Runtime:** `amlnn_toolkit` is an optional tool for converting other Framework models to quantized or floating-point TFLite models.

- **Buildroot System:**

Buildroot	amlnn_toolkit	Native C Compilation Environment
Native(c/c++ Api)	✓	✓
LiteRT Runtime	✓ (Optional)	×
Third-Party Runtime	×	×

- **LiteRT Runtime:** `amlnn_toolkit` is an optional tool for converting other Framework models to quantized or floating-point TFLite models.
- **Debian / Yocto System:**

Debian	<code>amlnn_toolkit</code>	<code>amlnn_edge_toolkit</code>	<code>amlnn_edge_toolkit_lite</code>	Native C Compilation Environment
Native(c/c++ Api)	✓ (Optional)	✓ (Optional)	×	✓
Native(python Api)	✓ (Optional)	✓ (Optional)	✓ (Optional)	×
LiteRT Runtime	✓ (Optional)	✓ (Optional)	×	×
Third-Party Runtime	×	×	×	×

- **Native(c/c++ Api):** Choose either `amlnn_toolkit` or `amlnn_edge_toolkit` to complete the conversion of supported Framework models to ADLA models.
- **Native(python Api):**
 - Method1: Directly use `amlnn_edge_toolkit` to complete ADLA model conversion, loading, and inference on the edge device. However, due to device memory limitations, some models may fail to convert.
 - Method2:
 - ADLA Model Conversion: Choose either `amlnn_toolkit` or `amlnn_edge_toolkit` to complete the conversion of supported Framework models to ADLA models.
 - ADLA Model Inference: Choose either `amlnn_edge_toolkit` or `amlnn_edge_toolkit_lite` to load and infer ADLA models.
- **LiteRT Runtime:** `amlnn_toolkit` / `amlnn_edge_toolkit` are optional tools for converting other Framework models to quantized or floating-point TFLite models.

2.2.2 Toolkit Installation

After selecting the integration method, if Toolkit installation is required, refer to this chapter for instructions. Miniforge is recommended for managing Python environments on both PC and Debian device sides.

2.2.2.1 Configuring Virtual Environment

- **Installation Package Download:**
 - x86_64: `wget https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-x86_64.sh`
 - aarch64: `wget https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Linux-aarch64.sh`
- **Miniforge Installation:**
 - `chmod +x Miniforge3-Linux-*.sh`
 - X86_64: `bash Miniforge3-Linux-x86_64.sh`
 - aarch64: `bash Miniforge3-Linux-aarch64.sh`

- **Verifying Installation:**

- `conda config --show channels` executed successfully, with `- conda-forge` in the result.

- **Creating Virtual Environment**

- Create virtual environment: `conda create -n amlnnn_toolkit python=3.10`
- Activate virtual environment: `conda activate amlnnn_toolkit`
- Python version can be adjusted as needed (3.10 recommended). 'amlnn_toolkit' is the virtual environment name and can be modified.

2.2.2.2 Installing Toolkit

After activating the `amlnn_toolkit` Conda environment, install via local wheel package.

Install:

Select the corresponding requirements and whl files based on the Python version and processor architecture.

```
# for amlnn_toolkit
```

```
Step1: Install dependencies one by one (to avoid version conflicts)
```

```
for req in $(cat requirements_x86_64.txt); do pip install $req; done
```

```
Step2: Install toolkit whl package
```

```
# toolkit-x.x.x is the toolkit version number, cpxx is the Python version tag  
(e.g., cp310, cp311)
```

```
pip install amlnn_toolkit-x.x.x-cpxxx-cpxxx-linux_x86_64.whl
```

```
# Note: Do NOT use `pip install -r requirements_x86_64.txt`, as it may cause  
dependency version conflicts. Install one by one.
```

```
# for amlnn_edge_toolkit
```

```
Step1: Install dependencies one by one (to avoid version conflicts)
```

```
for req in $(cat requirements_aarch64.txt); do pip install $req; done
```

```
Step2: Install toolkit whl package
```

```
# toolkit-x.x.x is the toolkit version number, cpxx is the Python version tag  
(e.g., cp310, cp311)
```

```
pip install amlnn_edge_toolkit-x.x.x-cpxxx-cpxxx-linux_aarch64.whl
```

```
# for amlnn_edge_toolkit_lite
```

```
pip install amlnn_edge_toolkit_lite-x.x.x-cpxxx-cpxxx-linux_aarch64.whl
```

Verify:

```
#Verify
```

```
# verify toolkit-x.x.x version
```

```
from amlnn.api import AMLNN
```

```
# verify toolkit_lite-x.x.x version
```

```
from amlnnlite.api import AMLNNLite
```

2.2.3 Native C Compilation Environment Installation

Installing the Native C compilation environment involves installing cross-compilation tools on the PC that correspond to the device's system. Different board system environments require different toolchains, as detailed below:

- **Android System:**
 - Cross-compilation tool download link: [Android NDK Download Address](#). `android-ndk-r25c-linux.zip` is recommended.
 - Extract to a specified directory, e.g., `/opt/android-ndk-r25c`.
 - Compilation Configuration
 - During compilation, specify `ANDROID_NDK_PATH` in `build-android64.sh` / `build-android32.sh` to the local toolchain path.
- **Buildroot System:**
 - Cross-compilation tool download links:
 - 32bit: [Buildroot 32 toolchain download link](#)
 - 64bit: [Buildroot 64 toolchain download link](#)
 - Extract to a specified directory, e.g.:
 - 32bit: `/opt/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi`
 - 64bit: `/opt/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu`
 - Compilation Configuration
 - During compilation, specify `BUILDROOT_TOOLCHAIN_PATH` in `build-buildroot64.sh` / `build-buildroot32.sh` to the local toolchain path.
- **Debian / Yocto System:**
 - Cross-compilation tool download links:
 - 32bit: [32 toolchain download link](#)
 - 64bit: [64 toolchain download link](#)
 - Execute the corresponding script:
 - 32bit: `./poky-glibc-x86_64-amlogic-bsp-armv7at2hf-neon-mesons7-bh201-5.15-a32-toolchain-4.0.20.sh`
 - 64bit: `./poky-glibc-x86_64-meta-toolchain-armv8a-mesont7-an400-5.15-a64-toolchain-4.0.20.sh`
 - Installation Options:
 - Choose installation path, e.g., `/opt/poky/4.0.20`.
 - Enter 'yes'.
 - Wait a moment, and the compilation toolchain will be installed.
 - Compilation Configuration
 - During compilation, specify `YOCTO_TOOLCHAIN_PATH` in `build-yocto64.sh` / `build-yocto32.sh` to the local toolchain path.

3 Model Conversion

3.1 Model Conversion Introduction

- AMLNN model conversion flow chart:

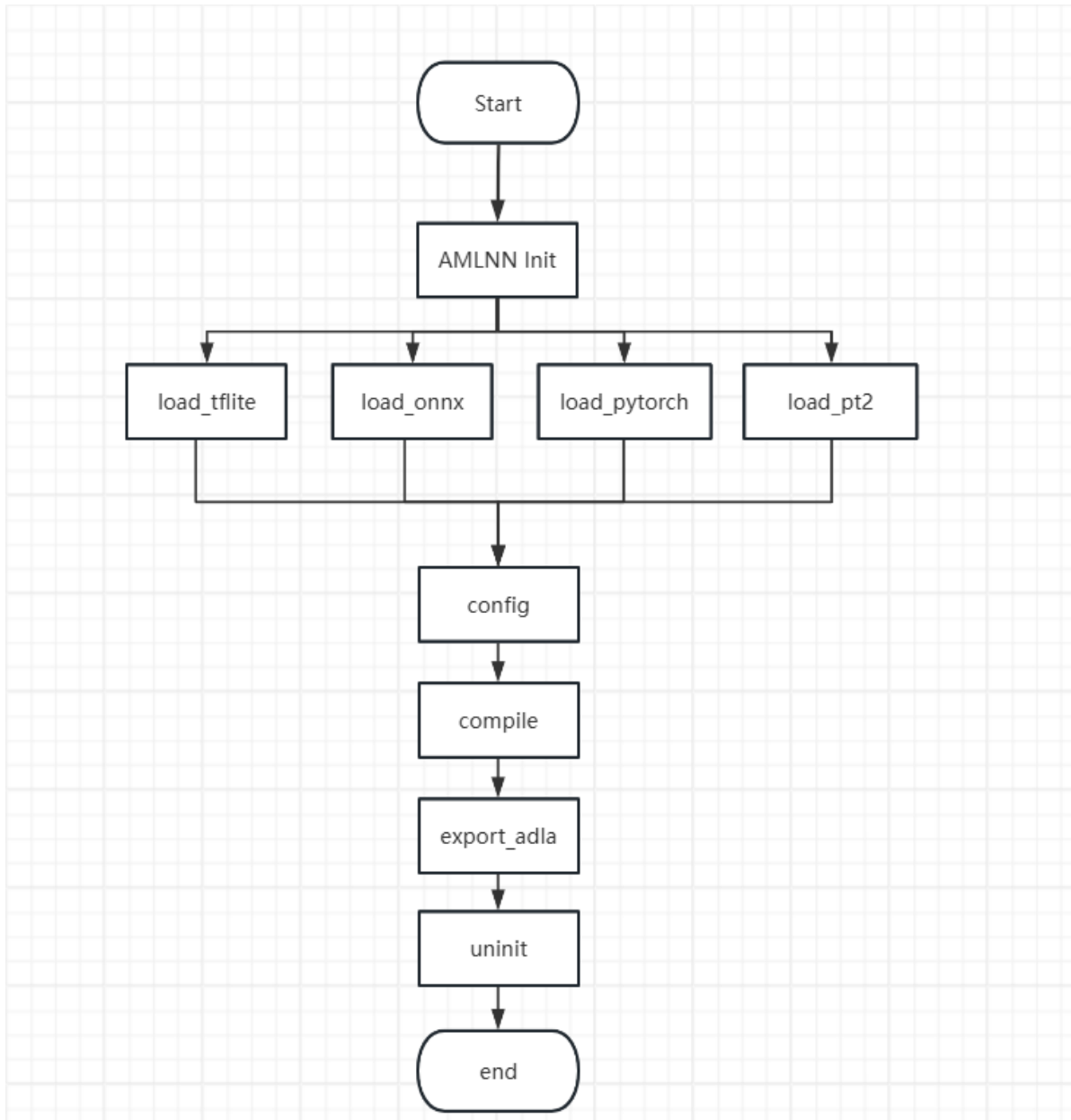


Figure 3.1-Model Conversion Flow Chart

- Framework versions supported by Amlnn-toolkit:

Framework	Version	Note
Tflite	1.15~2.8.0	
ONNX	1.9.1	
Pytorch(jit)	>=2.6.0	Torchscript Format (<code>torch.jit.trace</code> , <code>model.save</code>)

Framework	Version	Note
Pytorch(pt2)	>=2.6.0	Exported_program Format (<code>torch.export.save(exported_program, "model.pt2")</code>)
Tensorflow	TODO	
Paddle	TODO	

3.2 Model Conversion Preparation

3.2.1 Model Information Verification

- **Model Visualization Tool**

Netron tool can be used to visualize models and view their input/output information and structure.

- Netron tool links:

- Web version: <https://netron.app/>
- PC version: <https://sourceforge.net/projects/netron.mirror/files/v8.0.4/>. Download [Netron-Setup-8.0.4.exe](#) and double-click to install.

- **Model Visualization:**

Suitable for standalone models requiring data flow + weights. Taking PyTorch models as an example, three scenarios generally occur, as shown below:

- Left: Model file saved via `traced_model = torch.jit.trace() / traced_model.save('model.pt')`. Contains weights and data flow (arrows), supporting standalone deployment.
- Middle: Model file saved via `torch.save(model, 'model.pth')`. This model type is highly sensitive to code structure; most do not support standalone deployment. Not recommended.
- Right: Model file saved via `torch.save(model.state_dict(), 'weights.pth')`. Contains only weights, no data flow. Does not support standalone deployment.

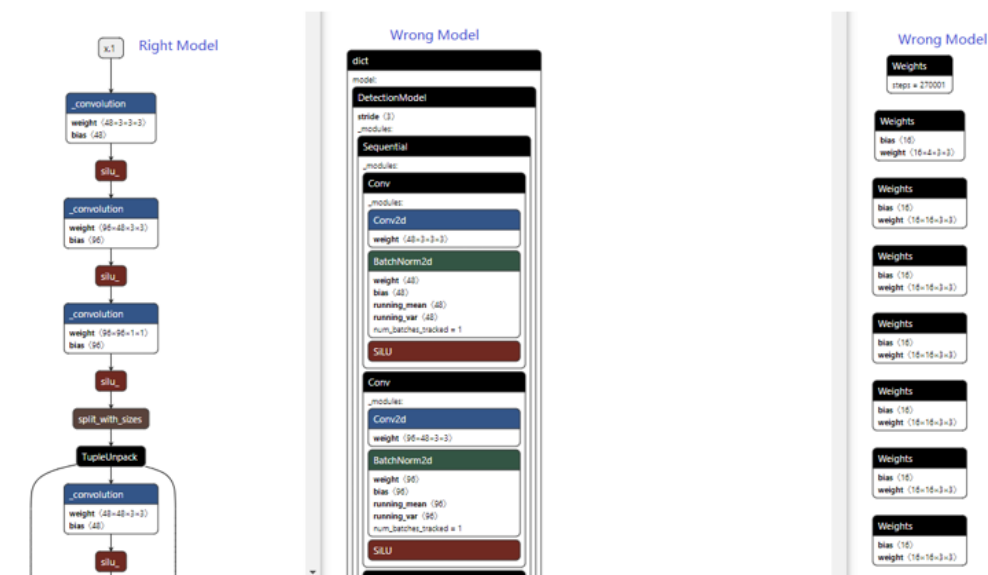
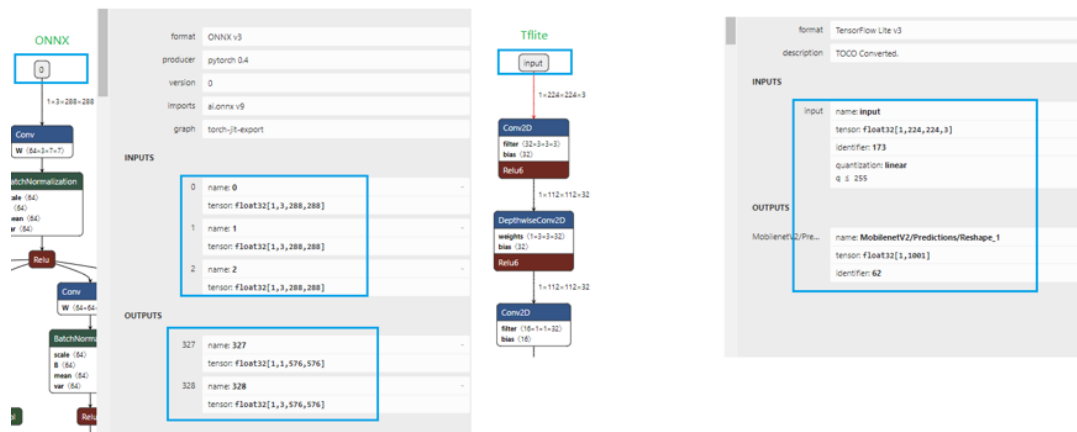


Figure 3.2-Model Visualization

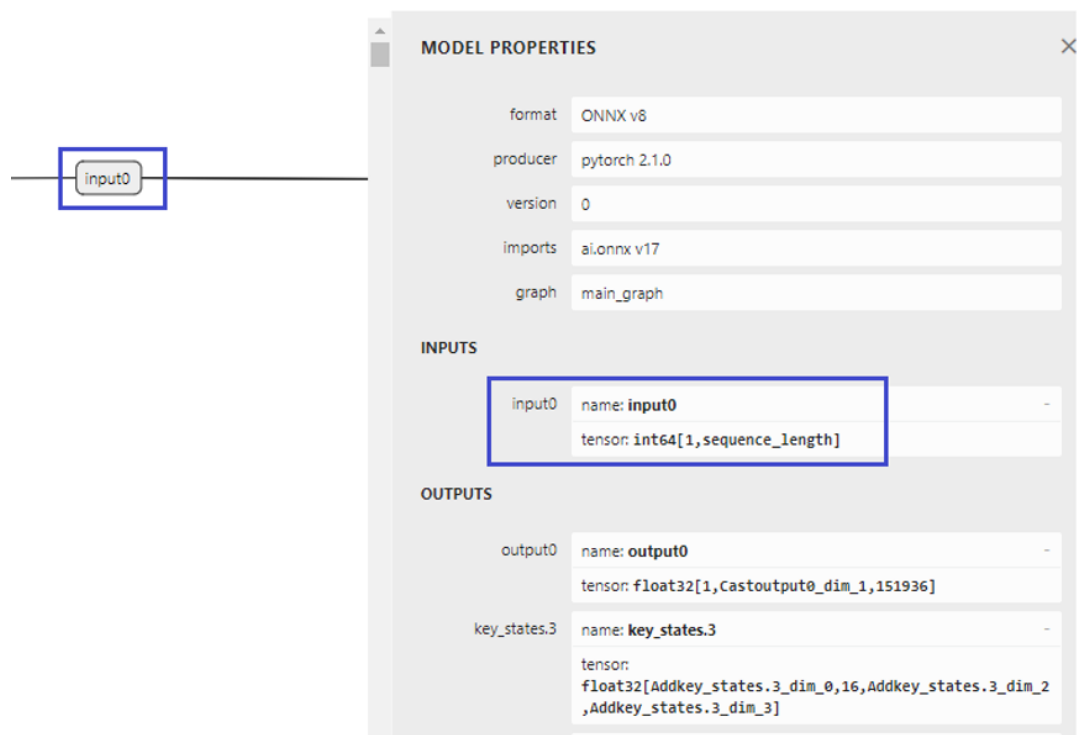
- **Model Input Information**

- **Onnx / Tflite models have fixed shape**



- **Onnx model without fixed shape**

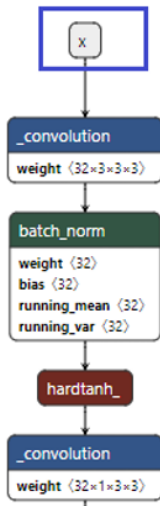
- During model conversion, parameters like `inputs / input_shapes` must be explicitly set.



- **Pytorch model without fixed Shape**

- During model conversion, parameters like `inputs / input_shapes / dtypes` must be explicitly set.

pytorch mobilenetv2



MODEL PROPERTIES

format TorchScript v1.6

INPUTS

x name: %x

OUTPUTS

%1688 name: %1688

3.2.2 Quantization Dataset Preparation

Currently, `amlnn_toolkit` supports two methods for providing quantization datasets: 1) Image dataset 2) NPY dataset. The image dataset is only suitable for models where the original training data source is images. In other cases, the NPY dataset is recommended.

- **Why a dataset is needed**

Currently, the A311Y3 and C305X2 platforms support all conversion types, while other platforms only support fixed-point operations like W8A8 and W8A16. When converting open-source framework models to fixed-point models, a floating-point to fixed-point quantization conversion is required—mapping the float min-max of each node during inference to the fixed-point value range. Therefore, a quantization dataset needs to be prepared. The quantization dataset consists of multiple sets of multi-dimensional arrays used for model quantization training or accuracy verification.

- **Image Dataset**

When using an image dataset, it is necessary to confirm the model preprocessing's `normalization_mean` and `normalization_std` information. Set parameters based on the preprocessing information used during model training. For the three-channel data (data1, data2, data3) after reading the raw image, range (0~255), the preprocessing flow is:

```
channel1 = (data1 - normalization_mean[0]) / normalization_std[0]
channel2 = (data2 - normalization_mean[1]) / normalization_std[1]
channel3 = (data3 - normalization_mean[2]) / normalization_std[2]
```

When using an image dataset, preprocessing parameters generally need to be set. Common preprocessing parameter references are as follows:

- Normalize to [-1,1]: `normalization_mean=[[127.5, 127.5, 127.5]], normalization_std=[[127.5, 127.5, 127.5]]`
- Normalize to [0,1]: `normalization_mean=[[0, 0, 0]], normalization_std=[[255, 255, 255]]`

- No preprocessing: `normalization_mean=[[0, 0, 0]]`, `normalization_std=[[1, 1, 1]]`
(In this scenario, mean and std parameters can be omitted).

For multi-input models, if different preprocessing parameters need to be set for multiple inputs, refer to:

```
normalization_mean=[[127.5, 127.5, 127.5], [0, 0, 0]], normalization_std=[[127.5, 127.5, 127.5], [255, 255, 255]]
```

- **NPY Dataset**

Suitable for scenarios where image datasets cannot be used, e.g., large language models, speech models, or other non-image input models. It is recommended that users complete preprocessing in the original project environment and save the final data as .npz files. When using the NPY dataset, there is no need to set the `normalization_mean` and `normalization_std` parameters.

```
# Reference code for saving npz

import torchvision.transforms as transforms

def preprocess(path):
    transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5])
    ])
    img = Image.open(path).convert('RGB')
    img = transform(img)
    img = img.transpose([1, 2, 0])
    img = np.expand_dims(img, axis=0).astype('float32')

    np.save("input_1.npz", img)
```

- **Quantization Dataset Organization**

A quantization dataset generally recommends providing 200~500 groups to ensure that the maximum/minimum values obtained during the quantization process are the actual maximum/minimum values from the runtime scenario. The dataset is set via the `dataset` parameter when calling the `amlnn.compile` interface, and the `normalization_mean` and `normalization_std` parameter information is set when calling the `amlnn.config` interface.

The content of the dataset file `dataset.txt` is as follows:

Single-input model:

```
1.jpg
2.jpg
or:
1.npz
2.npz
```

Multi-input model: (One set of inputs per line, separated by spaces)

```
input1_1.jpg input2_1.jpg
input1_2.jpg input2_2.jpg
or:
input1_1.npy input2_1.npy
input1_2.npy input2_2.npy
```

3.3 Model Conversion

3.3.1 AMLNN Initialization and Release

When using the API interfaces of Amlnn-toolkit, it is necessary to first call `AMLNN()` to initialize the AMLNN object, and call the `uninit()` method to release it upon completion. During initialization, set the log level via `log_level`. **Default value: 'INFO'**. Currently supports ['DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL']. The DEBUG level will print more conversion information.

```
# Example
from amlnn.api import AMLNN

amlnn = AMLNN(log_level='INFO')
...
amlnn.uninit()
```

3.3.2 Model Loading

`amlnn_toolkit` currently supports loading and converting models from ONNX, TFLite (float TFLite and quant TFLite), PyTorch (pt and pt2), etc. Different framework models require calling corresponding interfaces. The calling methods are as follows:

- **ONNX Model Loading**

- Use `amlnn.load_onnx()` to load an ONNX model.
- If the input shape of the ONNX model is not fixed, the `inputs` and `input_shapes` parameters need to be set explicitly.
- Supports truncating the ONNX model during the conversion process by setting the `outputs` parameter to the output tensor names of the truncation nodes.

```
# Example 1:

amlnn.load_onnx(model='./yolov8s_sim.onnx')

# Example 2: Load model and perform truncation

amlnn.load_onnx(model='./yolov8s_sim.onnx', \
    outputs=['/model.22/Concat_output_0', '/model.22/Concat_1_output_0', \
    '/model.22/Concat_2_output_0'])
```

- **PyTorch Model Loading**

- Use `amlnn.load_pytorch()` to load a PyTorch model. The model must be in TorchScript format.

- TorchScript format models generally lack input shape information, so parameters like `inputs`, `input_shapes`, `dtypes` need to be set to specify input-related information.
- If all model inputs are of float type, the `dtypes` parameter can be omitted (default value used). For multiple inputs, refer to: `dtypes=['float32', 'int32']`.

Example:

```
amlnn.load_pytorch(model='resnet18.pt',
                   inputs=['%x.2'],
                   input_shapes=[[1, 3, 224, 224]])
```

- **PT2 Model Loading**

- Use `amlnn.load_pt2()` to load a PyTorch pt2 model. The model must be in ExportedProgram format.
- Supports truncating the pt2 model during the conversion process by setting the `outputs` parameter to the output tensor names of the truncation nodes.

Example:

```
amlnn.load_pt2(model='yolov8n_exported.pt2')
```

- **TFLite Model Loading**

- Use `amlnn.load_tflite()` to load a TFLite model.
- Supports bypass import of TFLite and conversion to ADLA. Bypass is controlled by the parameter `quantized_model=True`.
- When the model itself is a quantized TFLite model (e.g., w4a8/w8a8), `quantized_model=True` must be explicitly set; otherwise, conversion errors will occur. For a float TFLite model, setting `quantized_model=True` will convert it to `f16*f16.adla`.

Example 1:

```
amlnn.load_tflite(model='yolov8n_sim.tflite')
```

Example 2:

```
amlnn.load_tflite(model='yolov8n_sim_w8a8.tflite', quantized_model=True)
```

- **TensorFlow Model Loading (TODO)**

- Use `amlnn.load_tensorflow()` to load a TensorFlow model.

- **Paddle Model Loading (TODO)**

- Use `amlnn.load_paddle()` to load a Paddle model.

- **Keras Model Loading (TODO)**

- Use `amlnn.load_keras()` to load a Keras model.

3.3.3 Model Conversion Parameter Configuration

Before model compilation and conversion, model conversion-related parameters need to be set via the `amlnn.config` interface to ensure the correctness of the conversion results. Common configuration parameters are as follows:

- **target_platform**: Specifies the platform type for generating the ADLA file. **Obtain the correct value based on Section 2.1.3: Platform Type Confirmation.** Currently supported types:

Part_number	target_platform
C308L / C302X	PRODUCT_PID0XA001
S928X	PRODUCT_PID0XA002
A311D2	PRODUCT_PID0XA003
T968D4	PRODUCT_PID0XA004
S905X5 / S905D5	PRODUCT_PID0XA005
C302X2	PRODUCT_PID0XA006
A311Y3	PRODUCT_PID0XA007
C305X2/A123X	PRODUCT_PID0XA008

- **quantized_dtype**: Specifies the quantization type. Different platforms support different quantization types. **Default value: 'w8a8'**. Currently supports: ['w4a8', 'w8a8', 'w4a16', 'w8a16', 'w16a16'].
 - Only PRODUCT_PID0XA007/PRODUCT_PID0XA008 platforms support all quantization types.
 - Other platforms only support: 'w8a8' and 'w8a16'.
- **activation_dtype**: Specifies the activation type. Only takes effect when **quantized_dtype='w8a16'**. **Default value: 'f16'**. Currently supports ['f16', 'bf16', 'i16'].
 - Use `activation_dtype` to specify the conversion type as int8 * int16 or int8 * f16 or int6*bf16.

target_platform	Weights * Feature	Support Params
PRODUCT_PID0XA001 ~ PRODUCT_PID0XA006	Int8*Int8	quantized_dtype='w8a8'
	Int8*Int16	quantized_dtype='w8a16', activation_dtype='i16'
PRODUCT_PID0XA007/PRODUCT_PID0XA008	Int4*int8	quantized_dtype='w4a8'
	Int8*Int8	quantized_dtype='w8a8'
	Int4*f16	quantized_dtype='w4a16' , activation_dtype='f16'
	Int4*bf16	quantized_dtype='w4a16' , activation_dtype='bf16'
	Int8*int16	quantized_dtype='w8a16', activation_dtype='i16'
	Int8*f16	quantized_dtype='w8a16', activation_dtype='f16'

target_platform	Weights * Feature	Support Params
	Int8*bf16	quantized_dtype='w8a16', activation_dtype='bf16'
	f16*f16	quantized_dtype='w16a16', activation_dtype='f16'
	bf16*bf16	quantized_dtype='w16a16', activation_dtype='bf16'

- **quantized_method**: Specifies the quantization method. **Default value: 'perchannel'**. Supports ['perlayer', 'perchannel']. `perchannel` means each channel of the convolution kernel has a different scaling factor. `perlayer` means the convolution kernel shares one scaling factor. `perchannel` generally offers better accuracy.
- **group_size**: Sub-channel-wise quantization, i.e., dividing the channels of fully connected layers into smaller subgroups for finer-grained precision control. **Default value: 32**. Currently supports: [32, 64, 128].
 - Only takes effect when `quantized_dtype` is W4A8 or w4a16.
 - Smaller `group_size` results in more quantization parameters and better accuracy.
 - Larger `group_size` results in fewer quantization parameters and better performance.
- **export_intermediate**: Specifies whether to export the intermediate IR as a TFLite model file. **Default value: False**.

```
python# Example:
# Corresponding conversion parameters:
# target_platform: PRODUCT_PID0XA007 (A311Y3), quantized_dtype: w8a8,
quantized_method: perchannel

amlnn.config(target_platform='PRODUCT_PID0XA007')
```

- The currently supported quantization conversions and their corresponding `weight_quant_algo` and `activation_quant_algo` are listed in the table below. For cases where `activation_dtype` is bf16, it is currently set via an environment variable as a temporary workaround: `export AMLNN_SET_FLOAT_DTYPE=bf16`. This will be unified under parameter control once the feature is verified. For detailed quantization conversion guidance, please refer to the upcoming document: *NPU Quantization Guide.pdf*

Format (weights*Feature)	quantized_dtype	activation_dtype	weight_quant_algo	activation_quant_algo
Int4*int8	w4a8	None	lsq	normal/histogram/omse/percentile
Int8*int8	w8a8	None	normal/lsq	normal/histogram/omse/percentile
Int4*f16	w4a16	f16	grq/lsq	/
Int4*bf16	w4a16	bf16	grq/lsq	/
Int8*int16	w8a16	i16	normal/lsq	normal/histogram/omse/percentile
Int8*f16	w8a16	f16	normal/lsq	/
Int8*bf16	w8a16	bf16	normal/lsq	/
f16*f16	w16a16	f16	/	/
bf16*bf16	w16a16	bf16	/	/

3.3.4 Model Compilation

After loading the model and configuring the parameters, users need to perform model quantization conversion via `amlnn.compile()`. Corresponding parameters are:

- **dataset**
 - : Quantization calibration dataset, specified via a `dataset.txt` file (containing paths to images or `.numpy` files).
 - **Default value: None**, uses random data for quantization, for performance testing only, does not guarantee accuracy. In this case, quantization algorithm parameters configured in the `config` interface are ineffective, and the default switches to min-max quantization.
 - When using `.numpy` files in `dataset.txt`, preprocessing must be completed before saving as `.numpy`, and the shape must match the `input_shapes` parameter. The `mean` / `std` parameters configured in `config` are not effective.
 - Dataset preparation refers to Section 3.2.2.
- **iterations**: Specifies the number of data samples (images or `.numpy` files) to be used for quantization, read from `dataset.txt`. **Default value: -1**, meaning iterate through all data; when `dataset` is `None`, only one set of random data is generated for quantization.

```
python# Example:
amlnn.compile(dataset='dataset.txt', iterations=200)
```

3.3.5 Exporting ADLA Model

After quantization conversion via `amlnn.compile()`, users export the ADLA model file via `amlnn.export_adla()`.

```
python# Example:
amlnn.export_adla()
```

3.3.6 Model Conversion Example

```
python# Example:
from amlnn.api import AMLNN

# Initialize amlnn
amlnn = AMLNN(log_level='INFO')

# Load onnx model
amlnn.load_onnx(model='resnet34-v1-7.onnx')

# Configure model conversion parameters
amlnn.config(normalization_mean=[[127.5, 127.5, 127.5]],
             normalization_std=[[127.5, 127.5, 127.5]],
             quantized_dtype='w8a8',
             target_platform='PRODUCT_PID0XA007')

# Model compilation
amlnn.compile(dataset='./dataset.txt')
```

```
# Export adla model file.  
#Upon successful execution, corresponding adla files will be generated in the  
`resnet34-v1-7_adla_unify` directory.  
amlnn.export_adla()
```

3.4 Model Evaluation

3.4.1 Model Inference

After completing ADLA model conversion, model validation and performance evaluation can be performed via **Board-connected Inference** or **Direct On-board Inference**. If the development environment only has a Debian/Yocto edge device, only **Direct On-board Inference** can be used. If it's a host+devices architecture (host is Linux X86/Arm PC, devices are edge devices with Amlogic NPU), use **Board-connected Inference**; however, when devices are Debian/Yocto edge devices, **Direct On-board Inference** can also be used. The two inference methods are described as follows:

Method 1: Direct On-board Inference

- Install the `amlnn_edge_toolkit` (or lite version) whl package on the board.
- Based on `amlnn_edge_toolkit` (or lite version), perform board-side inference via Python API.

Method 2: Board-connected Inference

Install `amlnn_toolkit` (or `amlnn_edge_toolkit`) on the host side, connect to the board via ADB protocol to transfer data and complete inference. This method relies on the `nn_server` service, which must be started on the board in advance.

Step 1: Obtain nnserver file

Obtain the nnserver file, link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/nnserver. Select the corresponding nnserver file based on the system environment confirmed in [2.1.2 System Environment and Architecture Confirmation].

- **Android 64-bit:** `amlnn_toolkit/nnserver/android/lib64/nnserver`
- **Android 32-bit:** `amlnn_toolkit/nnserver/android/lib32/nnserver`
- **Armbian/Yocto 64-bit:** `amlnn_toolkit/nnserver/linux/yocto/lib64/nnserver`
- **Armbian/Yocto 32-bit:** `amlnn_toolkit/nnserver/linux/yocto/lib32/nnserver`
- **Buildroot 64-bit:** `amlnn_toolkit/nnserver/linux/buildroot/lib64/nnserver`
- **Buildroot 32-bit:** `amlnn_toolkit/nnserver/linux/buildroot/lib32/nnserver`

Step 2: Start nnserver

```
adb shell "mkdir -p /data/nn"
adb push /path/to/nnserver /data/nn/
adb shell "chmod +x /data/nn/nnserver"
adb shell "cd /data/nn && ./nnserver"
```

If information similar to the following is displayed, the nnserver service has started successfully

```
NNSERVER, vx.x.x, 20xx.xx
Coordinator listening on port 8307
Server listening on port 8308
Server listening on port 8309
Server listening on port 8310
```

Main interfaces used during the model inference phase:

- Environment initialization interface `amlnn.init_runtime()` parameters:
 - **mode**: Operation mode, supports `native` and `'nnserver'`. Lite version only supports `'native'`.
 - **board_work_path**: Board-side working directory, storing intermediate files during operation. **Default value**: `/data/nn`.
 - **enable_perf**: Whether to enable performance collection. **Default: False**; must be set to `True` if performance information is required.
- Model loading interface `amlnn.load_model()` parameters:
 - **path**: ADLA model path. If `amlnn_toolkit` (or `amlnn_edge_toolkit`) has already been used to call `amlnn.export_adla` / `amlnn.init_runtime`, there is no need to explicitly call this interface again.
- Inference interface `amlnn.inference()` parameters:
 - **inputs**: Input data, can be a single `numpy.ndarray`, or a `list` / `tuple`.
 - **run_cycles**: Number of inference cycles. Default is 1.

```
import argparse
import numpy as np
from amlnn.api import AMLNN

def preprocess(image_path):
    pass

def postprocess(outputs):
    pass

# init amlnn
amlnn = AMLNN(log_level='INFO')

# load onnx model
amlnn.load_onnx(model='resnet34-v1-7.onnx')

# Configure model conversion parameters
amlnn.config(normalization_mean=[[127.5, 127.5, 127.5]],
             normalization_std=[[127.5, 127.5, 127.5]],
             quantized_dtype='w8a8',
             target_platform='PRODUCT_PID0XA007')
```

```

# compile model
amlnn.compile(dataset='./dataset.txt')

# export adla model file
amlnn.export_adla()

# Initialize environment configuration
ret = amlnn.init_runtime(mode='native', enable_perf=True)
if ret != 0:
    raise RuntimeError("init_runtime failed")

# Get sdk version
sdk_version = amlnn.get_sdk_version()
print(sdk_version)

# Get tensor info
tensor_info = amlnn.get_tensor_info()
print(tensor_info)

input_data = preprocess(image_path)

# Set input & Run inference & Get output
outputs = amlnn.inference(inputs=[input_data])

postprocess(outputs)

# Release resources
amlnn.uninit()

```

3.4.2 Model Performance Evaluation

Calling `amlnn.get_perf_info()` retrieves performance information for the most recent inference of the model, including NPU inference latency, frame rate, inference memory usage, bandwidth, total hardware/software operator time, etc. Two prerequisites must be met to use this interface:

- `enable_perf=True` was set when calling `amlnn.init_runtime()`.
- `amlnn.inference()` has been called.

After calling the `amlnn.get_perf_info()` interface, the return value is a `str`, a formatted performance information string, with fields including:

- `npu_avg_time_ms`: Average NPU inference time in milliseconds; when `run_cycles > 1`, returns the average across multiple cycles.
- `fps`: Frame rate calculated based on `npu_avg_time_ms`.
- `memory_usage_mb`: Memory usage required for the current model inference, in MB.
- `bandwidth_mb`: Total bandwidth read/write volume involved in the current model inference, in MB.
- `elapsed_in_hw_op_ms`: Total hardware operator execution time, in milliseconds.
- `elapsed_in_sw_op_ms`: Total software operator execution time, in milliseconds.

```

python# example

# init amlnn

```

```

amlnn = AMLNN(log_level='INFO')

# load onnx model
amlnn.load_onnx(model='resnet34-v1-7.onnx')

amlnn.config(normalization_mean=[[127.5, 127.5, 127.5]],
             normalization_std=[[127.5, 127.5, 127.5]],
             quantized_dtype='w8a8',
             target_platform='PRODUCT_PID0XA007')

# compile model
amlnn.compile(dataset='./dataset.txt')

# export adla model file
amlnn.export_adla()

# Initialize environment configuration
ret = amlnn.init_runtime(mode='native', enable_perf=True)

# Set input & Run inference & Get output
outputs = amlnn.inference(inputs=[input_data])

perf_info = amlnn.get_perf_info()
print(perf_info)

# Return value:
# npu_avg_time_ms=9.624, fps=103.907, memory_usage_mb=5.180, bandwidth_mb=28.802,
# elapsed_in_hw_op_ms=5.499, elapsed_in_sw_op_ms=0.000

```

3.4.3 Model Accuracy Evaluation

Currently, only adding post-processing logic after inference to verify model accuracy is supported. Future support for layer-by-layer accuracy comparison between ADLA models and intermediate IR (which can be exported as TFLite) is planned.

3.4.4 NPU Utilization

The script for checking NPU utilization is `NPU_utilization.py`. During the demo's runtime, open a new terminal and execute the following command to view CPU and NPU utilization in real-time.

Tool link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/function_tool/01_adla_get_utilization_tool

```
python NPU_utilization.py
```

The effect is shown in the figure below:



This script reads the NPU utilization node on the device at 50ms intervals. Non-zero utilization is only displayed when the sampling moment恰好 falls within the model's NPU inference window. For models with short inference times (e.g., MobileNet, ~2ms), most sampling points will show 0. To observe the actual NPU utilization more intuitively, the `--run-cycles` parameter in the demo script can be increased to make the model perform continuous inference multiple times, extending the inference window for easier capture by sampling.

3.5 On-device C API Inference

3.5.1 Process Introduction

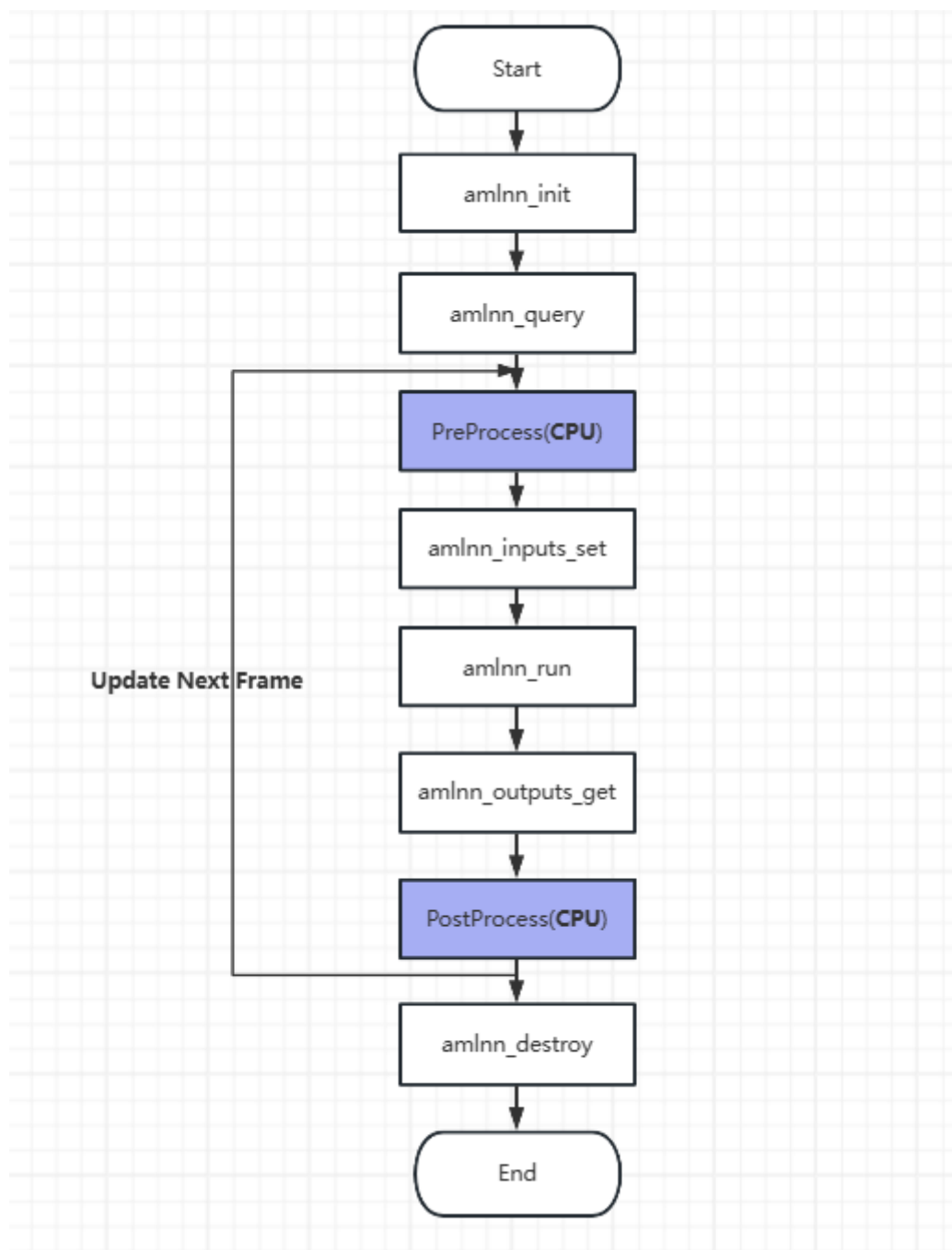


Figure 3.4.1 C-API Call Flow Chart

A general API call flow example code is as follows. Detailed interface usage reference: *NPU API Reference NNSDK2.pdf*.

```

cvoid *ctx = NULL;
amlnn_init_config init_config;
memset(&init_config, 0, sizeof(amlnn_init_config));

// init
ret = amlnn_init(&ctx, (void *)model_path, 0, &init_config);

// get input output number
amlnn_input_output_num io_num;
ret = amlnn_query(ctx, AMLNN_QUERY_IN_OUT_NUM, &io_num, sizeof(io_num));
printf("model input num: %d, output num: %d\n", io_num.n_input,
io_num.n_output);

// get input attr
amlnn_tensor_attr input_attrs[io_num.n_input];
memset(input_attrs, 0, sizeof(input_attrs));
for (uint32_t i = 0; i < io_num.n_input; i++)
{
    input_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_INPUT_ATTR, &(input_attrs[i]),
sizeof(amlnn_tensor_attr));
}

// get output attr
amlnn_tensor_attr output_attrs[io_num.n_output];
memset(output_attrs, 0, sizeof(output_attrs));
for (uint32_t i = 0; i < io_num.n_output; i++)
{
    output_attrs[i].index = i;
    ret = amlnn_query(ctx, AMLNN_QUERY_OUTPUT_ATTR, &(output_attrs[i]),
sizeof(amlnn_tensor_attr));
}

amlnn_input inputs[io_num.n_input];
amlnn_output outputs[io_num.n_output];
memset(inputs, 0, sizeof(inputs));
memset(outputs, 0, sizeof(outputs));

// PreProcess
input_data = PreProcess(image_path, xxxx);

// set input
memset(inputs, 0, sizeof(inputs));
inputs[0].index = 0;
inputs[0].buf = input_data; // Preprocessing including normalization,
quantization, data type matching, and data layout format matching must be done
externally. The SDK will directly copy this block of data to the NPU buffer.
inputs[0].size = input_size; // Input data size in bytes
ret = amlnn_inputs_set(ctx, io_num.n_input, inputs);

// Run adla
ret = amlnn_run(amlnn_ctx, nullptr);

// Get Output
ret = amlnn_outputs_get(amlnn_ctx, io_num.n_output, outputs, NULL);

```

```
// PostProcess  
post_process(outputs, results);  
  
amlnn_destroy(ctx);
```

4 On-device Python API Inference

amlnn_edge_toolkit_lite is a Python inference whl package developed by Amlogic specifically for Debian/Yocto systems on edge devices, facilitating AI demo development on edge devices using Python. It only contains the inference part of **amlnn_edge_toolkit**, supporting only the native method for direct on-device inference. Its interfaces are largely consistent with

`amlnn_edge_toolkit`. Usage requirements are as follows:

- On-device Python API requires the device environment to be Debian/Yocto system.
- ADLA files must first be exported via `amlnn_toolkit`.

4.1 Python API Process Introduction

All inference-related operations are performed via the `AMLNNLite` class. Currently, `AMLNNLite` only supports the `native` mode on the device.

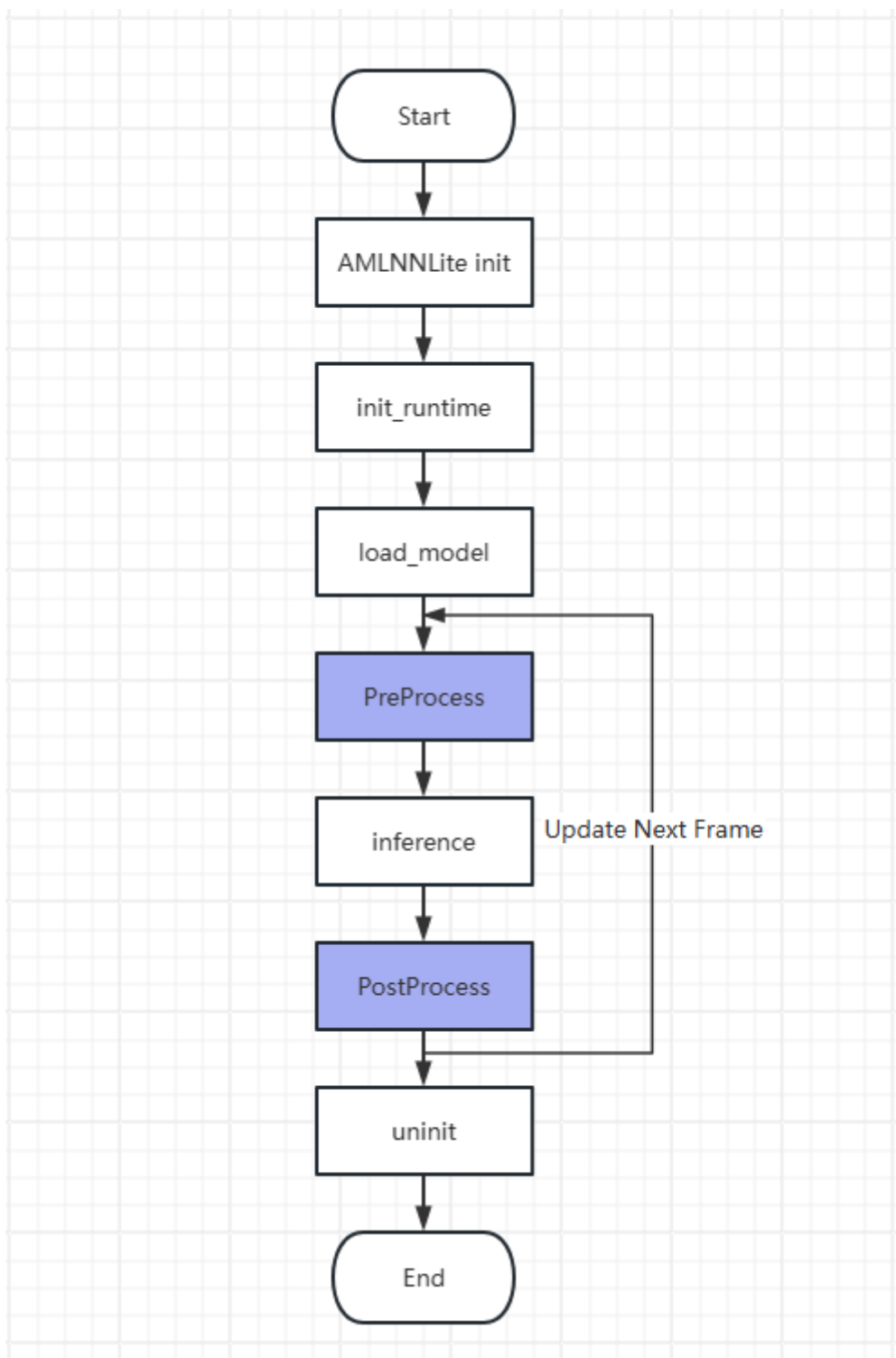


Figure 4.1 C-API Call Flow Chart

Example code:

```
pythonfrom amlnnlite.api import AMLNNLite

# init
amlnnlite = AMLNNLite()

# init runtime
amlnnlite.init_runtime()

# load adla
amlnnlite.load_model(path="xxx.adla")
```

```
# preprocess
input_data = PreProcess()

# inference
outputs = amlnnlite.inference(inputs=[input_data])

# PostProcess
PostProcess(outputs)

# uninit
amlnnlite.uninit()
```

4.2 Python API Description

4.2.1 AMLNNLite Initialization

When using the Amlnn-toolkit-lite API interfaces, it is necessary to first call `AMLNNLite()` to initialize the AMLNN object, and call the `uninit()` method to release it upon completion.

Function:

- Creates an `AMLNNLite` instance.
- Sets the log level on the Python side.

Parameter	Type	Description
log_level	str	Log level, supports <code>DEBUG</code> / <code>INFO</code> / <code>WARNING</code> / <code>ERROR</code> .

```
pythonfrom amlnnlite.api import AMLNNLite

amlnnlite = AMLNNLite(log_level="INFO")
```

4.2.2 init_runtime

Function:

- Initializes the runtime environment.
- In `AMLNNLite`, the `mode` only supports `native`.

Return Value: 0: Success, -1: Failure.

Parameter	Type	Description	Required
mode	str	Operation mode. Default value: 'native'. Lite version only supports <code>native</code> .	No
board_work_path	str	On-device working directory. Default value: <code>/data/nn</code> , stores intermediate files during operation.	No

Parameter	Type	Description	Required
enable_perf	bool	Whether to enable performance collection. Default value: False. Must be set to <code>True</code> if performance information is required.	No

```
ret = amlnnlite.init_runtime()
```

4.2.3 get_sdk_version

Function: Retrieves version information related to the current device-side SDK / ADLA / DELEGATE.

Return value: `str`, version string.

```
pythonsdk_version = amlnnlite.get_sdk_version()
print(sdk_version)
```

4.2.4 load_model

Function: Loads the runtime execution model.

Return Value: 0: Success, -1: Failure.

Parameter	Type	Description	Required
path	str	Local model path. Currently only <code>.adla</code> files are supported.	Yes

```
python
# Example

model_info = amlnnlite.load_model(path="./adla/resnet18-v1-7_int8_A311D2.adla")
```

4.2.5 get_tensor_info

Function: Retrieves input and output tensor information of the model.

Return value:

- dict, containing two fields:
 - `inputs`: Model input tensor information list.
 - `outputs`: Model output tensor information list.

```
#python

pythontensor_info = amlnnlite.get_tensor_info()
print(tensor_info)
```

Each tensor's information is represented as a `dict`, with field descriptions as follows:

Field Name	Type	Description
index	int	The index of the current tensor in the input or output list.
name	str	Tensor name.
n_dims	int	Number of tensor dimensions.
dims	tuple	Native shape of the tensor, corresponding in order to <code>format_name</code> .
n_elems	int	Total number of tensor elements.
size	int	Original effective data size of the tensor, in bytes.
size_with_stride	int	Buffer size with alignment, in bytes.
format	int	Internal enumeration value of the tensor layout format.
format_name	str	Tensor layout format name, e.g., <code>NHWC</code> / <code>NCHW</code> .
type	int	Internal enumeration value of the tensor data type.
type_name	str	Tensor data type name, e.g., <code>INT8</code> , <code>UINT8</code> , <code>FP32</code> , etc.
zp	int	Quantization zero point.
scale	float	Quantization scale.

4.2.6 inference

Function: Executes model inference.

Return value: Returns a list of output tensors.

- `List[np.ndarray]`
- The returned output tensor data is in `float32` format.

Parameter	Type	Description	Required
inputs	list/tuple/np.ndarray	Input data, can be a single <code>numpy.ndarray</code> , or a <code>list</code> / <code>tuple</code> .	Yes
inputs_data_format	str	Input data format, supports <code>NHWC</code> / <code>NCHW</code> . Default value is <code>NHWC</code> .	No
outputs_data_format	str	Output data format, supports <code>NHWC</code> / <code>NCHW</code> . Default value is <code>NHWC</code> .	No
run_cycles	int	Number of inference cycles. Default value is <code>1</code> .	No

```
#python

pythonoutputs = amlnnlite.inference(
    inputs=[input_data],
    inputs_data_format="NHWC",
    outputs_data_format="NHWC",
    run_cycles=1,
)
```

4.2.7 uninit

Function: Releases runtime and model resources.

```
#python

amlnnlite.uninit()
```

4.2.8 Python API Example Demo

```
pythonimport argparse
import numpy as np
from amlnnlite.api import AMLNNLite

def preprocess(image_path):
    pass

def postprocess(outputs):
    pass

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--model-path", default="./model_file/xxx.adla")
    parser.add_argument("--image-path", default="./input/xxx.jpg")
    args = parser.parse_args()

    # Initialize AMLNNLite
    amlnnlite = AMLNNLite(loglevel="ERROR")

    # Initialize environment configuration
    ret = amlnnlite.init_runtime(enable_perf=False)
    if ret != 0:
        raise RuntimeError("init_runtime failed")

    # Get sdk version
    sdk_version = amlnnlite.get_sdk_version()
    print(sdk_version)

    # Load model
    amlnnlite.load_model(path=args.model_path)

    # Get tensor info
    tensor_info = amlnnlite.get_tensor_info()
    print(tensor_info)
```

```
input_data = preprocess(args.image_path)
# Set input & Run inference & Get output
outputs = amlnnlite.inference(inputs=[input_data])

postprocess(outputs)
# Release resources
amlnnlite.uninit()

if __name__ == "__main__":
    main()
```